

Dijkstra's Algorithm

Introduction

Dijkstra's algorithm is a method for finding the shortest path between nodes in a weighted graph. It was invented by computer scientist Edsger W. Dijkstra in 1956 and published in 1959. The algorithm finds the shortest path from a single source node to every other node in a graph. It can also be stopped early if only the path to one specific destination node is needed.

A common way to think about this algorithm is through the example of a road network. If the nodes of a graph represent cities and the edges represent roads with distances attached to them, Dijkstra's algorithm can be used to find the shortest route between one city and every other city. This is exactly the kind of problem solved by GPS navigation systems and mapping applications every day.

The algorithm is also used in computer networking. Two well known routing protocols, IS-IS and OSPF, use a form of Dijkstra's algorithm to determine the best paths for data to travel across a network. It is also used as a building block inside other algorithms, such as Johnson's algorithm, which finds shortest paths between all pairs of nodes in a graph.

History

Dijkstra came up with the idea for this algorithm in 1956 while working as a programmer at the Mathematical Center in Amsterdam. In an interview many years later, he explained that he designed the algorithm in about twenty minutes while sitting at a cafe with his fiancée. He was trying to think of a way to demonstrate the abilities of a new computer called ARMAC to people who were not familiar with computing.

He chose the shortest path problem because it was easy to explain and easy to check by hand. He implemented the algorithm using a simplified map of sixty four cities in the Netherlands. He chose the number sixty four because it could be represented using exactly six bits of memory, which was convenient for the computer he was working with.

Dijkstra did not publish the algorithm until 1959, three years after he first came up with it. He mentioned that he designed it without using pencil and paper, which he later felt was actually helpful, because it forced him to keep the algorithm as simple as possible and avoid unnecessary complexity.

Interestingly, around the same time, Dijkstra also worked on a separate but related problem: how to connect all the pins on a computer's back panel using the least amount of wire. To solve this, he ended up rediscovering an algorithm for finding a minimum spanning tree, which is now known as Prim's algorithm. This algorithm had actually been discovered earlier by a mathematician named Jarnik, and was independently discovered again later by Prim, whose name it carries today.

How the Algorithm Works

The basic idea behind Dijkstra's algorithm is to always process the closest unvisited node first, and use it to try to find shorter paths to its neighboring nodes. The algorithm keeps track of two main pieces of information for every node: its current best known distance from the starting node, and which node came before it on that shortest path.

At the very beginning, the algorithm creates a set containing all the nodes in the graph, referred to as the unvisited set. Every node is assigned a distance value. The starting node is given a distance of zero, since the distance from the starting node to

itself is obviously zero. Every other node is given a distance of infinity, because at this point, no path to them has been discovered yet.

The algorithm then enters a loop that continues until every node has been visited, or until the destination node has been reached, if only a single destination is needed. In each pass of the loop, the algorithm looks at all of the nodes still in the unvisited set and selects the one with the smallest known distance value. This node becomes the current node.

Once the current node has been selected, the algorithm looks at each of its neighboring nodes that are still unvisited. For each neighbor, the algorithm calculates what the distance to that neighbor would be if you traveled through the current node. This is done by adding the current node's distance to the weight of the edge connecting it to the neighbor.

If this newly calculated distance is smaller than the neighbor's previously recorded distance, the neighbor's distance is updated to this smaller value, and the current node is recorded as the neighbor's predecessor. This process of checking and possibly updating a neighbor's distance is often referred to as relaxing an edge. After all of the current node's neighbors have been checked, the current node is removed from the unvisited set, since its shortest distance is now considered final and will never change again. The algorithm then repeats the process, again selecting the unvisited node with the smallest distance value.

This process continues until there are no more nodes left in the unvisited set, or until all of the remaining unvisited nodes have a distance of infinity, meaning they cannot be reached from the starting node at all. When the algorithm finishes, every node that was reachable from the starting node will have its correct shortest distance recorded.

A Simple Manual Example

Imagine a small map with five intersections labeled A, B, C, D, and E, connected by roads of different lengths. Suppose you are trying to find the shortest path from intersection A to every other intersection.

At the start, intersection A is labeled with a distance of zero, and all other intersections are labeled with infinity. Intersection A becomes the current node. Its neighbors are checked one by one, and their distances are updated based on the road lengths connecting them to A.

Once all of A's neighbors have been checked and updated, A is marked as visited and removed from further consideration. The algorithm then looks at all the unvisited intersections and selects the one with the smallest current distance value. This becomes the new current node, and the same process of checking its neighbors is repeated.

This continues, intersection by intersection, until every reachable intersection has been visited. At the end of this process, every intersection has a final distance value representing the shortest possible distance from intersection A, along with a record of which intersection comes right before it on that shortest path.

By following these predecessor records backward from any destination intersection, you can reconstruct the actual shortest route from A to that destination, not just its total distance.

Using a Priority Queue

Although the basic version of Dijkstra's algorithm can be implemented using a simple list of nodes, this becomes slow for large graphs, because searching through the entire unvisited set to find the node with the smallest distance takes a long time as

the graph grows.

To make the algorithm faster, a data structure called a priority queue is commonly used instead. A priority queue allows the algorithm to quickly retrieve the node with the smallest distance value without having to search through the entire set every single time.

Several types of priority queues can be used for this purpose, including binary heaps, pairing heaps, and Fibonacci heaps. Each of these offers different performance characteristics, but all of them are significantly faster than a simple linear search once the graph becomes large.

When using a basic array or list to represent the unvisited set, the algorithm has a running time of $O(V^2)$, where V represents the number of vertices, or nodes, in the graph. This is because for every node processed, the algorithm may need to scan through all other nodes to find the one with the smallest distance.

When using a binary heap as the priority queue, the running time improves significantly to $O((E + V) \log V)$, where E represents the number of edges in the graph. For graphs where the number of edges is much smaller than the square of the number of vertices, this is a large improvement.

Using a more advanced structure called a Fibonacci heap, the running time can be further improved to $O(E + V \log V)$. This is generally considered the fastest known running time for Dijkstra's algorithm on graphs with arbitrary non-negative edge weights, although more specialized algorithms exist for certain special cases, such as graphs with only integer weights.

Limitations of the Algorithm

One important limitation of Dijkstra's algorithm is that it only works correctly when all of the edge weights in the graph are non-negative. If a graph contains negative edge weights, Dijkstra's algorithm can produce incorrect results, because it assumes that once a node has been visited, its shortest distance has been finalized and will never need to be updated again. Negative edge weights can violate this assumption.

For graphs that may contain negative edge weights, a different algorithm called the Bellman-Ford algorithm should be used instead. The Bellman-Ford algorithm is somewhat slower than Dijkstra's algorithm, but it is able to correctly handle negative edge weights, and it can even detect the presence of negative weight cycles in a graph, which represent situations where no shortest path actually exists.

Another limitation is that the basic version of Dijkstra's algorithm computes the shortest path from a single source node to every other node in the graph, even if you are only interested in the path to one specific destination. Although the algorithm can be stopped early once the destination node has been reached, it still may explore a large portion of the graph before reaching that point, especially if the destination is far from the source.

Real World Applications

Dijkstra's algorithm has many practical applications outside of pure computer science theory. One of the most well known applications is in GPS navigation systems, which use variations of Dijkstra's algorithm to calculate the shortest or fastest route between two locations on a road network.

In computer networking, routing protocols such as OSPF, which stands for Open Shortest Path First, use Dijkstra's algorithm to determine the most efficient paths for data packets to travel across a network of routers. Each router calculates the shortest path to every other router in the network, allowing data to be forwarded efficiently.

Dijkstra's algorithm is also commonly used in video games, particularly for pathfinding, where computer controlled characters need to determine the shortest route from one point on a map to another while avoiding obstacles.

In addition, Dijkstra's algorithm serves as the foundation for several other more advanced algorithms. For example, the A* search algorithm, which is widely used in artificial intelligence and robotics, is essentially an extension of Dijkstra's algorithm that uses additional information to guide the search more efficiently toward a specific goal.

Conclusion

Dijkstra's algorithm remains one of the most important and widely taught algorithms in computer science, more than sixty years after it was first invented. Its combination of simplicity, correctness, and efficiency has made it a fundamental tool for solving shortest path problems in graphs, with applications ranging from everyday navigation systems to complex computer networks.